

The background is a dark blue gradient with a subtle pattern of white dots. On the left side, there are several concentric circles and a large circular scale with degree markings from 140 to 260. Some of the circles have arrows indicating a clockwise direction. The title text is positioned on the right side of the image.

TRANSFER LEARNING WITH CIFAR 10

AUTHOR: GIANCARLO LATERZO

DATA LOADING AND HYPERPARAMETERS

Data Loading:

- Loading the Raw Train/Test Folders from the CIFAR10 Dataset
- Creating a PATH where the best checkpoints are saved

```
1 raw_data_train = '/content/drive/MyDrive/Machine Learning/Datasets/  
CIFAR-10-images-master/train'  
2  
3 raw_data_test = '/content/drive/MyDrive/Machine Learning/Datasets/  
CIFAR-10-images-master/train'  
4  
5 PATH = "/content/drive/MyDrive/"
```

Hyperparameters:

- Learning rate of 0.0001
- Batch Size of 64
- Training over 10 Epochs
- Variable for the number of classes (10 in the CIFAR10)

```
1 lr = 1e-4  
2 BATCH_SIZE = 64  
3 N_EPOCHS = 10  
4 NUM_CLASSES = 10
```


MODEL (VISION TRANSFORMER)

- Loaded **ViT-B/32** with pretrained *ImageNet* weights
- Replaced original 1k class head with `nn.Linear()`
- Initialized new head weights with **Xavier Uniform**
- The backbone stays trainable (not frozen)

```
# Model (Visual Transformer)
model = models.vit_b_32(weights=models.ViT_B_32_Weights.DEFAULT)

model.heads = torch.nn.Linear(
    in_features=model.heads.head.in_features,
    out_features=NUM_CLASSES
)

torch.nn.init.xavier_uniform_(model.heads.weight)
print(model)

model.train()
```

TRAINING LOOP

Adam Optimizer

- **Backbone:** 0.0001 ($1e-4$)
- **Head:** 0.001 ($1e-3$)

CrossEntropyLoss: Label Smoothing of 0.1 prevents overconfidence

Cosine LR Scheduler: Smoothly decays the learning rate over 10 epochs

```
def training_loop(N_EPOCHS, model, loss_fn, opt, scheduler):

    best_f1 = 0.0

    for epoch in range(N_EPOCHS):
        model.train()

        for xb, yb in train_dl:
            xb = xb.to(torch_device)
            yb = yb.to(torch_device)

            y_pred = model(xb)
            loss = loss_fn(y_pred, yb)

            opt.zero_grad()
            loss.backward()
            opt.step()

        scheduler.step()

        # End of Epoch Evaluation
        model.eval()

        all_preds, all_targets = [], []

        with torch.no_grad():
            for xb, yb in test_dl:
                xb = xb.to(torch_device)
                yb = yb.to(torch_device)
                preds = torch.argmax(model(xb), dim=1)
                all_preds.extend(preds.cpu().numpy())
                all_targets.extend(yb.cpu().numpy())

        epoch_f1 = f1_score(all_targets, all_preds, average='macro')

        if epoch > best_f1:
            best_f1 = epoch_f1
            torch.save(model.state_dict(), PATH + "best_weights.pt")

        print(f"Epoch: {epoch}\n Loss: {loss.item():.4f}\n Epoch F1: {epoch_f1:.4f}")

    print(f"Best F1 Score: {best_f1:.4f}")
```

EVALUATION

F1-Score (Macro): 0.9439

F1-Score: 0.9439860522877316

Confusion Matrix:

```
Confusion Matrix:
[[962  1  8  7  3  1  1  0 12  5]
 [ 3 970  1  1  0  0  0  2  7 16]
 [10  0 909 28 22 15  7  5  2  2]
 [ 2  1  4 903 19 53  9  3  5  1]
 [ 0  0  3 11 961  7  4 14  0  0]
 [ 2  0  4 77 13 895  3  6  0  0]
 [ 3  0  4 14  7  5 965  0  1  1]
 [ 4  0  0  6 15 18  1 954  2  0]
 [11  3  2  3  0  1  1  0 977  2]
 [ 3 26  1  7  1  2  0  4 14 942]]
```